Repaso para la prueba

Hoja de una pasada de la **Unidad 1 (semanas 1 a 4)**: el mapa, el hilo de «leer un botón», la chuleta, los errores de siempre y 18 ejercicios con respuestas al final.

Hardware · Software · código · Regla · decisión · Error · ojo · ◇ decisión

1 Mapa de la unidad

cada estación = una guía

1	**S1·1 Arquitectura**	CPU, buses, memorias
2	**S1·2 Bits y registros**	máscaras, DDR / PORT / PIN
3	**S2·1 ESP32**	GPIO, W1TS / W1TC, pines, superlazo
4	**S2·2 Programar para el hardware**	tipos, `millis()`, pulsador
5	**S3 Temporizadores**	prescaler, alarma, ISR
6	**S4 Interrupciones externas**	`attachInterrupt`, IRAM, `volatile`, bandera; hereda el antirrebote de S2·2 y la ISR de S3

2 «Leer un botón», de la S1 a la S4

el hilo de toda la unidad

1	**S1 · registro AVR**	leer el pin directo	`PIND & (1 << 2)`
2	**S2 · sondeo**	`digitalRead` + `millis()`: antirrebote y flanco	
3	**S3 · el hardware avisa solo**	ISR + `volatile` + bandera	
4	**S4 · `attachInterrupt`**	FALLING y antirrebote dentro de la ISR	

CHULETA (DE MEMORIA)

BITS [02]	CÓDIGO
Encender bit n	`REG |= (1 << n);`
Apagar bit n	`REG &= ~(1 << n);`
Invertir bit n	`REG ^= (1 << n);`
Leer bit n	`if (REG & (1 << n))`

DDR	PORT	PINES AVR [02]
0	0	Entrada flotante
0	1	Pull-up (suelto 1, pulsado 0)
1	0	Salida 0 V
1	1	Salida 5 V

Pesos: 128 64 32 16 8 4 2 1 · 1 dígito hex = 4 bits · `0x20` = bit 5.

ESP32 [03]

- DDR → `GPIO_ENABLE_REG` · PORT → `W1TS` / `W1TC` · PIN → `GPIO_IN_REG`. **W1TS/W1TC = atómico.**
- **Seguros:** 16–19, 21–23, 25–27, 32, 33 · **Strapping:** 0, 2, 5, 12, 15 · **Solo entrada:** 34–36, 39 · **UART0:** 1, 3 · **Flash:** 6–11.
- **3,3 V, no tolera 5 V** · ≤ 10 mA por pin · `R = (3,3 – Vf) / I`.
- Arranque: ROM → 2.º bootloader → `app_main` → `loopTask` → `setup()` → `loop()`.

MEMORIAS [01]

Flash = programa (no se borra) · **SRAM** = variables (se borra) · **EEPROM** = datos que sobreviven (100 000 ciclos; nunca escribir en `loop()`).

TEMPORIZADORES [05]

- `f = 80 000 000 / N`
- `resolución = 1 / f`
- `tiempo = alarma / f`
- `alarma = tiempo × f`
- N = 80 → 1 tick = 1 µs → **alarma en µs.** ESP32: 4 timers de 64 bits; AVR: 3 de 8/16 bits.

TIPOS Y TIEMPO [04]

- `int`: 2 bytes en el Uno, 4 en el ESP32 → `uint8_t`, `uint32_t`.
- `if (ahora - tAnterior >= PERIODO)`, tiempos en `uint32_t`.
- `1UL << 31`, no `1 << 31` · `(uint8_t)~a` vuelve a 8 bits.
- Sin signo da la vuelta; con signo, indefinido.
- Pulsador: `INPUT_PULLUP` + antirrebote 20–50 ms + **flanco**.

INTERRUPCIONES [05][06]

- Evento → ISR corta con `IRAM_ATTR` → `volatile bool bandera = true` → `loop()`: la baja y trabaja.
- `attachInterrupt(digitalPinToInterrupt(pin), isr, FALLING);` en `setup()`, **después** de `pinMode`; ISR **sin paréntesis**.
- Pull-up + «al presionar» = **FALLING**. CHANGE = dos veces por toque.
- ISR: ✔ bandera, `millis()`, contador, `digitalWrite` · ✘ `Serial`, `delay`, cálculos largos (→ **watchdog**).
- `volatile` solo si la tocan **ISR y loop**; no es atómico. La bandera no cuenta: para contar, `volatile uint32_t pulsos`.

ERRORES QUE SE PREGUNTAN SIEMPRE

ERROR	POR QUÉ ESTÁ MAL	CORRECTO
`REG = (1 << 2);`	Borra los demás bits	`REG |= (1 << 2);`
`int t = millis();`	Desborda	`uint32_t t = millis();`
`millis() > t + P`	La suma da la vuelta	`millis() - t >= P`
`for (uint8_t i = 0; i < 256; i++)`	Bucle infinito	`uint16_t i`
`delay(200)` como antirrebote	Bloquea el programa	Antirrebote con `millis()`
`if (digitalRead(B) == LOW) n++;`	Cuenta por nivel	Detectar el flanco
LED en GPIO34	Solo entrada	Pin seguro (16–19…)
`INPUT_PULLUP` en GPIO35	Sin pull-up interna	Pull-up externa 10 kΩ
Sensor de 5 V directo al ESP32	No tolera 5 V	Divisor o adaptador de nivel
EEPROM escrita en `loop()`	Se desgasta en minutos	Escribir solo cuando cambia
`Serial.println` en la ISR	Watchdog reinicia	Patrón bandera
Variable compartida sin `volatile`	El loop no ve el cambio	`volatile`
ISR sin `IRAM_ATTR`	Falla de vez en cuando	`void IRAM_ATTR isr()`
Timers de la versión 2.x	No compila en core 3.x	API 3.x: `timerBegin(freq)`

3 18 ejercicios

toca una pregunta para ir a su respuesta

01 Convierte 0xC8 a binario y decimal. [02]

02 ¿Qué vale 0xCA & 0x0F? [02]

03 Escribe la línea que invierte el bit 3 de PORTB. [02]

04 DDRD2 = 0, PORTD2 = 0, nada conectado. ¿Qué lee PIND2? [02]

05 ¿Por qué W1TS/W1TC son más seguros que `GPIO_OUT |= mask`? [03]

06 ¿Por qué un pulsador entre GPIO12 y 3,3 V puede impedir que arranque la placa? [03]

07 Resistencia para un LED rojo (Vf = 2,0 V) a 4 mA en el ESP32. [03]

08 `uint8_t c = 253;` y se imprime `c++` cinco veces. ¿Qué sale? [04]

09 `uint8_t x = 200, y = 100; uint8_t r = x + y;` ¿Cuánto vale r? [04]

10 ¿Qué imprime la 3.ª vuelta con una global, una local y una static que se incrementan? [03]

11 Aviso cada 250 ms con ticks de 1 µs. ¿Prescaler y alarma? [05]

12 f = 10 000 y alarma = 5 000. ¿Cada cuánto? ¿Qué N? [05]

13 ¿Cuánto tarda en llenarse un contador de 16 bits con ticks de 1 µs? [05]

14 Diferencia entre autorrecarga y desbordamiento. [05]

15 Diferencia entre interrupción del temporizador y externa. [05][06]

16 Tres toques con el loop en `delay(2000)` y bandera `bool`: ¿cuántos avisos procesa? [06]

17 ¿Qué falla si olvidas volatile? ¿Y si olvidas IRAM_ATTR? [06]

18 ¿Por qué Harvard puede ser más rápida que von Neumann? [01]

RESPUESTAS · 18 EJERCICIOS

01 `1100 1000` = 128 + 64 + 8 = **200**. ↩

02 `1100 1010 & 0000 1111 = 0000 1010` = **0x0A**. ↩

03 `PORTB ^= (1 << 3);` ↩

04 Un valor **al azar**: entrada flotante. ↩

05 Escriben **sin leer antes**: una sola escritura atómica. Leer-modificar-escribir pierde el cambio de una interrupción (u otro núcleo) en medio. ↩

06 GPIO12 es de **strapping**: si está en alto al arrancar, la flash se configura a 1,8 V y el chip no arranca. ↩

07 R = (3,3 − 2,0) / 0,004 ≈ **325 Ω** → 330 Ω. ↩

08 `253 254 255 0 1` (sin signo da la vuelta). ↩

09 **44**: se calcula como `int` (300) y al guardar en 8 bits queda 300 − 256. ↩

10 `global=3 local=1 static=3`. ↩

11 N = 80 000 000 / 1 000 000 = **80**; alarma = 0,25 × 1 000 000 = **250 000**. ↩

12 5 000 / 10 000 = **0,5 s**; N = 80 000 000 / 10 000 = **8 000**. ↩

13 65 535 µs ≈ **65,5 ms**. (64 bits: ≈ 584 000 años.) ↩

14 Autorrecarga: vuelve a 0 en la **alarma que eliges** (uso normal). Desbordamiento: en el **máximo físico** de bits (casi nunca intencional). ↩

15 Temporizador: periférico interno, **predecible**. Externa: un **pin**, **sin aviso**. Ambas usan ISR, IRAM_ATTR, volatile y bandera. ↩

16 **Uno**: la bandera dice «pasó algo», no cuántas veces. Para contar: `volatile uint32_t pulsos`. ↩

17 Sin `volatile`: el loop **nunca ve** la bandera. Sin `IRAM_ATTR`: con la caché apagada la ISR no se puede leer y el chip **se cae** de vez en cuando. ↩

18 Dos memorias y dos buses: trae la siguiente instrucción mientras usa un dato. Von Neumann comparte un bus y va por turnos. ↩